lecture 14
dynamic programming

Recall Strategies

Make It A Little Better Each Time
- selection, insertion,

Divide and Conquer
- make each half great then combine. merge sort

Do slow things infrequently
- amortized time is good. dynamic array
- other operations are fast

Reduce, It to a graph algorithm
- apply graph algorithm

Dynamic Programming
Greedy Strategies
} combinatorial optimization problem
- given a sequence of decisions, make the best one

Example Problem: What is the best way to break a paragraph into lines

typesetting: put words into lines. where to put the breaks

Constraints: 1. line width: characters per line
2. can't just put 1 character/word per line
3. goal: minimize trailing spaces. underflow
4. don't care about trailing space on last line
5. better for extra space to be evenly distributed.
- cost: is the extra space squared
- prefer evenly distributed spaces

Algorithm: works on any cost function

```
def line_cost(line_width, line, is_last):
    width = 0
    for word in line:
        width += len(word) + 1

    width -= 1                      no space for last line
    if width > line_width:
        return ∞
    if is_last:
        return 0
    return (line_width - width)^2
```

Ideas: You can not assume greedy will work

make it a little better each time. greedy
- fill up a line as much as possible
- do for each line
- never revisit earlier decision
- first decision is always made before other decisions

example
width: 12
greedy output. greedy tends to have fast run time

a l i c e w a s b e g i n n i n g t o g e t v e r y t i r e d

0	1	2	3	4	5	6	7	8	9	10	11	remaining
a	l	i	c	e		w	a	s				3
b	e	g	i	n	n	i	n	g				0
t	o		g	e	t		v	e	r	y		4
t	i	r	e	d								0

lines	cost
1	$3^2 = 9$
2	$0^2 = 0$
3	$4^2 = 16$
4	—
	25

at each step, make the logically optimal choice
very fast. O(n) time. loop through and insert words

Is this the optimal way?

0	1	2	3	4	5	6	7	8	9	10	11	remaining
a	l	i	c	e		w	a	s				3
b	e	g	i	n	n	i	n	g				3
t	o		g	e	t		v	e	r	y		1
t	i	r	e	d								0

lines	cost
1	$3^2 = 9$
2	$3^2 = 9$
3	$1^2 = 1$
4	—
	20

Greedy strategy does not guarantee optimal solution

Con: a local best decision but put us in a bad state in the future
Could not determine effect of previous moves

Obvious greedy strategy don't work
need to prove greedy is correct

needs a way to compute the effect of the cost each decision has on the rest of the problem
- use recursion

Trick: we need to compute the effect on the cost of every possible choice for the rest of the problem. use recursion

Combinatorial Optimization Algorithm

Strategy with recursion
- loop over every possible option for 1st choice. try every place for that 1st break
- use recursion to determine the optimal cost condition of making that choice
- pick best one of those out of all options

```
def opt_lines(line_width, words) -> best_soln, best_cost:
    best_soln ∈ [words]          # initial choice
    best_cost ∈ line_cost(line_width, words, True)

    # loop over all possible split points
    for split in 1, 2, ..., len(words)-1:
        line ∈ [words[0], ..., words[split-1]]
        rest ∈ [words[split], ..., words[len(words)-1]]

        cost_of_line ∈ line_cost(line_width, line, False)
        rest_soln, rest_cost ∈ opt_lines(line_width, rest)
        cost ∈ cost_of_line + rest_cost

        if cost < best_cost:
            best_soln ∈ [line] + rest_soln
            best_cost ∈ cost
    return best_soln, best_cost
```

consider remaining lines

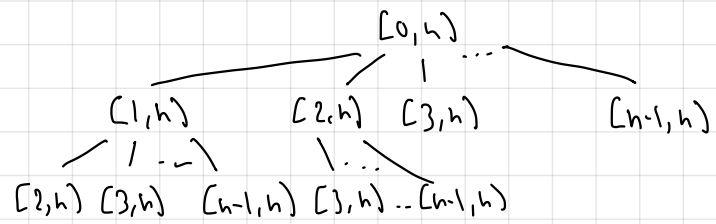
use recursion

recall selection sort loops through options and picks the best one

assume all words are less than the length of a line

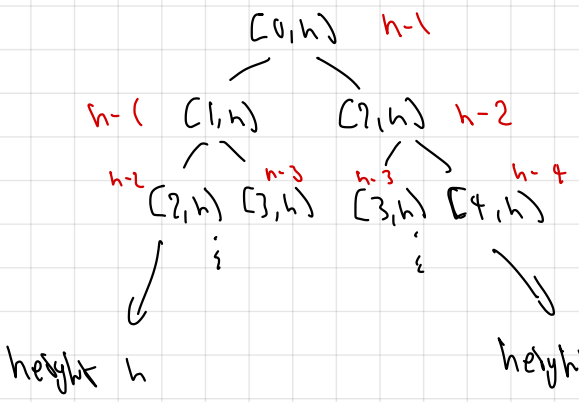
---

recursion tree



$C_n = \sum_{i=1}^{n-1} C_i$

lower bound: just take first two children



at least $\frac{h}{2}$ levels. each level has at least 2 nodes

at least $2^{\frac{h}{2}}$ nodes.

$\geq 2^{\frac{h}{2}}$ nodes

$\geq \sqrt{2}^{h}$ nodes

same analysis as AVL lower bound runtime

big O is at least exponential $2^{\frac{h}{2}}$

insight: lots of redundant work
lots of equivalent subtrees

---

how many unique inputs:

$[0,n), [1,n), [2,n), \ldots, [n-1,n)$

$(n-1) - 0 + 1 = n$ unique inputs

the first branch does all of the unique calls

after that, it's just repeats.

Changes
- Introduce global map called memo
- any time you compute something, store it in the memo table
- before any computation, check memo table

```
def opt_lines(line_width, words) -> best_soln, best_cost:

  if (line_width, words) in memo:
      return memo[(line_width, words)]

  best_soln ← [words]
  best_cost ← line_cost(line_width, words, True)

  for split in 1, 2, ..., len(words)-1:
      line ← [words[0], ..., words[split-1]]
      rest ← [words[split], ..., words[len(words)-1]]

      cost_of_line ← line_cost(line_width, line, false)
      rest_soln, rest_cost ← opt_lines(line_width, rest)
      cost ← cost_of_line + rest_cost

      if cost < best_cost:
          best_soln ← [line] + rest_soln
          best_cost ← cost

  memo[(line_width, words)] ← best_soln, best_cost

  return best_soln, best_cost
```

Reduces computation to only nodes that have unique inputs

n unique inputs
n^2 work per call. n loop. each loop calls line_cost, which is n. n·n

$O(n^3)$ reduced exponential to polynomial

Additional Speedup
- line_cost can be optimized, making total run time of optimal $O(n^2)$